

Class6: R functions

Rowen (PID: A18363990)

Background

All functions in R have at least 3 things:

- a name (we pick that and use it to call the function)
- a input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function),
- the *body* (the lines of R code that do the work of the function).

A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either “**required**” or “**optional**”. The later have fall-back default values that will be used if the user does not specify them.

```
add <- function(x,y=1000) {  
  x + y  
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 1010
```

Q1b: For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7,10))`. [1 pt]

```
add <- function(a, b = 0) {  
  sum(a, b)  
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add(c(4, 7, 10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the `...` (dots) argument or the base R function `sum()`. [2 pts]

```
add <- function(...) {  
  sum(...)  
}
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line by using the `return()` function call.)

A `generate_dna()` function

A useful function here is the “base R” `sample()` function:

```
sample(x=c("A", "C", "G", "T"), size=10, replace = TRUE)
```

```
[1] "A" "A" "A" "A" "A" "T" "A" "A" "C" "T"
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

Q2. Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user.

Q2a. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(len) {  
  sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)  
}  
  
#test  
generate_dna(6)
```

```
[1] "C" "T" "C" "T" "A" "T"
```

```
generate_dna()
```

```
[1] "A" "T" "A" "A"
```

```
generate_dna(len=100)
```

```
[1] "T" "C" "C" "C" "G" "A" "G" "G" "T" "G" "T" "C" "T" "A" "C" "G" "A" "G"  
[19] "C" "A" "T" "C" "G" "C" "G" "C" "A" "T" "C" "G" "G" "A" "T" "T" "G" "T"  
[37] "C" "T" "C" "A" "G" "G" "G" "T" "G" "C" "T" "T" "C" "C" "C" "C" "A" "C"  
[55] "C" "A" "G" "A" "A" "G" "C" "T" "C" "T" "A" "G" "T" "A" "A" "A" "C" "T"  
[73] "G" "C" "A" "C" "G" "T" "T" "G" "A" "G" "G" "A" "G" "T" "C" "G" "A" "A"  
[91] "G" "G" "T" "A" "A" "A" "G" "C" "C" "T"
```

Q2b. Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

```
generate_dna <- function(len=10, single.element = FALSE) {
  dna <- sample(c("A", "C", "G", "T"), size=len, replace = TRUE)

  if(single.element) {
    #cat("is it me you are looking for...")
    dna <- paste(dna, collapse = "")
  }

  return(dna)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()` and `return()`

```
generate_dna(10)
```

```
[1] "G" "C" "T" "G" "T" "G" "A" "T" "T" "A"
```

```
generate_dna(10, single.element = TRUE)
```

```
[1] "TTGTTGTCCA"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
len9 CGAAGGCTG
```

```
generate_dna <- function(len = 10) {
  dna <- sample(c("A", "C", "G", "T"), size = len, replace = TRUE)

  dna_string <- paste(dna, collapse = "")

  cat(paste0(">len", len, "\n"))
  cat(dna_string, "\n")

  ##Format as FASTA with an ID line
  ##cat( paste(">len", len, "\n", sep="" ))
  ##cat(ans)
  ##cat("\n")
  ##
}
```

```

    return(dna_string)
}

```

```
x <- generate_dna(16)
```

```

>len16
CTAACAGTTATTGGTT

```

Write a generate_protein() function

Q3. Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG". [2 pts]

```

generate_protein <- function(len = 9) {
  # Create a vector of the 20 standard amino acids
  aa <- c("A","R","N","D","C","E","Q","G","H","I",
          "L","K","M","F","P","S","T","W","Y","V")

  ans <- sample(x=aa, size=len, replace = TRUE)
  paste(ans, collapse = "")
}

```

```
generate_protein(5)
```

```
[1] "QPSSQ"
```

Generate random protein sequences of length 6 to 13

Q4. Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand. [2 pts]

```

for(l in 6:13) {
  cat(">", l, "\n", sep="")
  cat( generate_protein(l), "\n")
}

```

```
>6
LQSHPH
>7
INIWGFP
>8
LNDKKATT
>9
NSSILKLTFF
>10
IRDLEKHHHPN
>11
VPCIYYMQPHN
>12
GHILNWPYIGCQ
>13
LSAQVRLFWIMAS
```

```
generate_protein(6)
```

```
[1] "MNKTFF"
```

```
generate_protein(7)
```

```
[1] "QRTWRVK"
```

```
generate_protein(8)
```

```
[1] "IQVSTCFSS"
```

```
generate_protein(9)
```

```
[1] "VWFSMLDFG"
```

Are Our peptides “unique in nature”?

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

length	Ide	Cov	Unique
6	100	100	N
7	100	100	N
8	100	100	N
9	100	78	Y
10	90	100	Y
11	81.82	100	Y
12	83.33	92	Y
13	88.89	69	Y

Q5a: At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)? [1 pt]

Peptides start to look unique in nature around length 9. This is the first length where there is no longer a 100% identity and 100% coverage match, which means the sequence is no longer an exact match to anything in the database.

Q5b: Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^n for a peptide of length n) and to the size of the known protein universe. [1 pt]

Very short peptides are almost always found because the number of possible sequences is small, so many of them already exist in known proteins. As the length increases, the number of possible sequences grows larger while the known protein database is limited, which makes the longer sequences less likely to match.

Connecting our findings to immunology

Q6: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

MHC class II probably presents peptides that are about 9 amino acids or longer because since that’s when they start to look unique based on the data. Short peptides show up everywhere, so they can’t really tell the difference between your own proteins and foreign ones. As peptides get longer, there are way more possible combinations, so they’re more likely to be unique. If the immune system used really short peptides, it might mistake your own cells as foreign. So using longer peptides helps it see more accurately.